

计算机系统基础 — PA2

一、实验实现流程

1. 引言

在 PA1 中，我们完成了 NEMU 的基础功能，完整地搭建出了一台简单的图灵机 TRM。然而真正的计算机系统远不止于此：它还需要支持丰富的 x86 指令，需要标志寄存器来协助实现条件分支等功能，同时还能使用 I/O 设备与外界交互。在 PA2 中，我们将循序渐进地为 NEMU 补齐这些能力，使其具备可以运行现代程序的能力。

整体上，PA2 分为三个递进的阶段。阶段一以最简单的 C 程序 dummy 为目标，在 NEMU 中实现 EFLAGS 寄存器、若干 RTL 伪指令以及 call、push、pop、sub、xor、ret 六条 x86 指令，让一个完整的函数调用与返回流程能在 NEMU 中正确执行。阶段二以 cputest 测试集为目标，按需扩充指令集，依托 RTL 与译码器解耦的设计实现各类算术、逻辑、移位、数据传送与控制转移指令；同时实现 differential testing，在每条指令执行后将 NEMU 的寄存器状态与 QEMU 比对，作为基础设施的一部分，帮助我们更方便地定位指令错误。阶段三则转向输入输出，实现 in、out 端口指令和内存映射 I/O 的分发逻辑，并在 AM 层面补全 _putc、_uptime、_read_key、_draw_rect 等 API，使 NEMU 能驱动串口、时钟、键盘与 VGA 显存，最终使其可以正常通过 hello、timetest、keytest、videotest 等测试程序，并在 Dhrystone、CoreMark、microbench 上完成跑分。

接下来将依照以上的三个阶段，逐步讲解其实现流程。

2. 阶段一：运行 dummy

在 PA2 的第一阶段，我们主要实现运行 `nexus-am/tests/cputest/tests/dummy.c` 这一最为基础的测试程序。dummy 程序只有一个空的 `main` 函数，但是从编译的角度来看，却包含了完整的函数调用与返回流程：从 `_start` 经 `_trm_init` 跳到 `main`，再原路返回到 `_halt`。

在具体实现的过程中，我们在调试模式下在 NEMU 中运行这一测试程序，观察其报错与反汇编文件，逐步补全实现每个报错的环节，最终使得 dummy 可以正常运行通过测试。现在我们可以知道，要让 NEMU 完整地运行 dummy 这一测试程序，需要实现 call、push、sub、xor、pop、ret 这六条 x86 指令，以及它们所依赖的整套基础设施。接下来将逐步实现这一流程

i. 添加 EFLAGS 寄存器

x86 的 EFLAGS 是一个 32 位的标志寄存器, 其中每一位都有特定的语义。在 NEMU 中, 我们只需关心 CF、ZF、SF、IF、OF 五个标志, 但即便不使用其他位, 它们的物理位置也必须保留, 以便后续实现 differential testing 时能与 QEMU 逐位对齐。一种自然的实现方式是利用 C 语言的位域, 将每个标志放在对应的 bit 位置上, 并通过 union 暴露一个 32 位整型视图, 使整寄存器既可以按位读写, 也可以一次性读出。其具体定义如下:

```

1  union {
2      struct {
3          uint32_t CF : 1;
4          uint32_t    : 5;
5          uint32_t ZF : 1;
6          uint32_t SF : 1;
7          uint32_t    : 1;
8          uint32_t IF : 1;
9          uint32_t    : 1;
10         uint32_t OF : 1;
11         uint32_t    : 20;
12     };
13     uint32_t val;
14 }eflags;
```

完成上述结构后, 还需在 `restart()` 中进行初始化。依照 i386 手册规定, bit 1 作为保留位, 此处将 `cpu.eflags.val` 初始化为 `0x2`。

ii. RTL 指令

NEMU 用 RTL (寄存器传输语言) 描述每条 x86 指令的语义。实验框架中已经实现了 `rtl_add`、`rtl_sub`、`rtl_lm`、`rtl_sm` 等基本指令, 剩下的部分则需要我们使用基本指令来实现。我们首先完成不依赖 EFLAGS 的工具函数, 包括数据移动、位提取、栈访问与比较:

```

1  static inline void rtl_mv(rtlreg_t* dest, const rtlreg_t *src1) {
2      *dest = *src1;
3  }
4
5  static inline void rtl_not(rtlreg_t* dest) {
6      *dest = ~*dest;
7  }
8
```

```

9 static inline void rtl_sext(rtlreg_t* dest, const rtlreg_t* src1, int
    width) {
10     switch (width) {
11         case 1: *dest = (int32_t)(int8_t)(*src1); break;
12         case 2: *dest = (int32_t)(int16_t)(*src1); break;
13         case 4: *dest = *src1; break;
14         default: assert(0);
15     }
16 }
17
18 static inline void rtl_push(const rtlreg_t* src1) {
19     cpu.esp -= 4;
20     vaddr_write(cpu.esp, 4, *src1);
21 }
22
23 static inline void rtl_pop(rtlreg_t* dest) {
24     *dest = vaddr_read(cpu.esp, 4);
25     cpu.esp += 4;
26 }
27
28 static inline void rtl_msb(rtlreg_t* dest, const rtlreg_t* src1, int
    width) {
29     *dest = (*src1 >> (width * 8 - 1)) & 0x1;
30 }

```

其中 `rtl_sext` 借助了 C 的整型提升规则：先把位宽较小的整数显式转换为有符号类型，再转为 32 位变量，进行隐式转换。

接下来则是依赖 EFLAGS 实现的访问器。依照实验框架，借助宏 `make_rtl_setget_eflags` 一次性生成 CF、OF、ZF、SF 四组 `rtl_set_*` 与 `rtl_get_*` 函数：

```

1 #define make_rtl_setget_eflags(f) \
2     static inline void concat(rtl_set_, f) (const rtlreg_t* src) { \
3         cpu.eflags.f = *src; \
4     } \
5     static inline void concat(rtl_get_, f) (rtlreg_t* dest) { \
6         *dest = cpu.eflags.f; \
7     }
8
9 make_rtl_setget_eflags(CF)
10 make_rtl_setget_eflags(OF)
11 make_rtl_setget_eflags(ZF)

```

```
12 make_rtl_setget_eflags(SF)
```

因为位域成员只占 1 个 bit，赋值时编译器会自动屏蔽高位，写入 0 或非零都会被规整到 0/1 之间，这恰好与标志位的语义相契合。在此之上，`rtl_update_ZF` 与 `rtl_update_SF` 根据指令的操作数宽度对运算结果做掩码后再更新标志，确保 8 位、16 位、32 位运算可以共享同一份代码。

iii. 实现所需的指令

完成了上述的基础部分，我们可以着手进行指令的实现了。这六条所需的指令的函数分布在 `data-mov.c`、`control.c`、`arith.c`、`logic.c` 这几个文件中。借助前面搭好的 RTL，每条指令的实现都可以以一种较为简便的形式来实现，例如 push 与 pop：

```
1 make_EHelper(push) {
2     rtl_push(&id_dest->val);
3     print_asm_template1(push);
4 }
5
6 make_EHelper(pop) {
7     rtl_pop(&t0);
8     operand_write(id_dest, &t0);
9     print_asm_template1(pop);
10 }
```

call 与 ret 的关键在于借助 `decoding.is_jump` 与 `decoding.jump_eip` 通知 `update_eip()` 进行随机跳转；call 在跳转前要把 call 指令之后的那条指令的地址（即 `decoding.seq_eip`）压入栈中作为返回地址，ret 则完全相反，会从栈顶将这个地址弹出。

```
1 make_EHelper(call) {
2     rtl_li(&t0, decoding.seq_eip);
3     rtl_push(&t0);
4     decoding.is_jump = 1;
5     print_asm("call %x", decoding.jump_eip);
6 }
7
8 make_EHelper(ret) {
9     rtl_pop(&t0);
10    decoding.jump_eip = t0;
11    decoding.is_jump = 1;
12    print_asm("ret");
```

```
13 }
```

sub 是这一阶段相对复杂的指令，因为它要同时更新 CF、OF、ZF、SF 四个标志。CF 用无符号比较 `rtl_sltu` 检测被减数是否小于减数(产生借位); OF 则要判断两个操作数符号是否不同且结果的符号也与被减数不同,这里使用 `(dest ^ src) & (dest ^ result)` 的最高位来实现。

```
1 make_EHelper(sub) {
2     rtl_sub(&t2, &id_dest->val, &id_src->val);
3     operand_write(id_dest, &t2);
4
5     rtl_sltu(&t0, &id_dest->val, &id_src->val);
6     rtl_set_CF(&t0);
7     rtl_update_ZFSF(&t2, id_dest->width);
8
9     rtl_xor(&t0, &id_dest->val, &id_src->val);
10    rtl_xor(&t1, &id_dest->val, &t2);
11    rtl_and(&t0, &t0, &t1);
12    rtl_msb(&t0, &t0, id_dest->width);
13    rtl_set_OF(&t0);
14
15    print_asm_template2(sub);
16 }
```

xor 则属于逻辑指令：CF 与 OF 直接清零，ZF、SF 正常更新以反映运算结果。实际上，这一模式对所有逻辑指令通用，后续阶段实现 and、or、test 时，也只需替换核心运算即可。

iv. 注册到 opcode_table

函数写好之后,还需要在 `opcode_table` 中注册对应的 opcode,并在 `all-instr.h` 中给出声明。

这里我们以 push 与 pop 为例，两者的寄存器形式分别占用 `0x50-0x57` 与 `0x58-0x5f` 共 16 个 opcode 的槽位，分别对应八个寄存器：

```
1 /* 0x50 */ IDEX(r, push), IDEX(r, push), IDEX(r, push), IDEX(r, push),
2 /* 0x54 */ IDEX(r, push), IDEX(r, push), IDEX(r, push), IDEX(r, push),
3 /* 0x58 */ IDEX(r, pop), IDEX(r, pop), IDEX(r, pop), IDEX(r, pop),
4 ,
```

```

4  /* 0x5c */  IDEX(r, pop),  IDEX(r, pop),  IDEX(r, pop),  IDEX(r, pop)
    ,

```

寄存器编号由译码函数 `decode_op_r` 直接从 opcode 的低 3 位提取，因此八个位置填同一组 `IDEX` 即可。sub 的立即数形式属于 0x83 扩展组 gp1 的子项 /5，因此还要在 gp1 子表中将 `EX(sub)` 填到 /5 的位置；call、jmp 的 rel32 形式则是在 0xe8 与 0xe9。

在实验框架中，需要实现的函数框架已经给出，这里我们先把其他需要的所有.c 文件中的函数在 `all-instr.h` 中进行声明，以免后续实现过程中在头文件上的反复修改。

```

1  // dummy
2  make_EHelper(push);
3  make_EHelper(pop);
4  make_EHelper(call);
5  make_EHelper(ret);
6  make_EHelper(jmp);
7  make_EHelper(sub);
8  make_EHelper(xor);
9  // add
10 make_EHelper(add);
11
12 // data-mov.c
13 make_EHelper(pusha);
14 make_EHelper(popa);
15 make_EHelper(leave);
16 make_EHelper(cltd);
17 make_EHelper(cwtl);
18 make_EHelper(movsx);
19 make_EHelper(movzx);
20 make_EHelper(lea);
21
22 // arith.c
23 // .....
24
25 // .....

```

完成了以上的内容后，在 NEMU 中运行 dummy，输出 `HIT GOOD TRAP` 通过了测试，PA2 阶段一完成。

在后续的过程中，也是通过相似的流程逐步实现，一边调试一边补全所需的代码块。经过几轮的运行测试与代码编写，最终实现 cputest 中的所有测试程序。

3. 实现完整的 cputest

在完成了 dummy 这部分之后，我们继续实现剩余的测试程序。查看 cputest，其中有如下文件：

```

1 dev@ubuntu:~\$ ls ~/ics2017/nexus-am/tests/cputest/tests/
2 add.c      bubble-sort.c  fib.c          if-else.c      matrix-mul.c
3 mov-c.c    pascal.c       recursion.c    shuixianhua.c  sum.c
4 unalign.c  add-longlong.c dummy.c         goldbach.c     leap-year.c
5 max.c      movsx.c        prime.c        select-sort.c  string.c
6 switch.c   wanshu.c       bit.c          fact.c         hello-str.c
7 load-store.c          min3.c         mul-longlong.c quick-sort.c
8 shift.c    sub-longlong.c to-lower-case.c

```

依照指导书要求，我们将按照 其它 → string → hello-str 的次序进行测试与实现。但是在此之前，为了应对后续可能出现的各种 bug，我们先来实现 differential testing 这一部分的功能作为基础设施，而后再进行后续的指令实现等流程。

i. 实现 diff-test

differential testing 的核心思想是让 NEMU 与 QEMU 同时执行同一个程序，每执行完一条指令就比较两者的寄存器状态：一旦发现不一致，立即停下报告。它的重要性在于可以尽可能提前地把潜在的指令错误暴露出来，使错误的传播被控制在一条指令的范围内，从而大幅降低 debug 难度。

在实验框架中，已经把通信和初始化部分准备好了。框架通过 fork 一个子进程来启动 QEMU，并利用 GDB 协议与之通信，向 QEMU 发送单步、读寄存器、写寄存器等命令；初始化阶段，将一段固定的实模式启动代码（MBR）拷入 QEMU 的 0x7c00 处，让 QEMU 进入保护模式后停在与 NEMU 相同的入口；每次 `cpu_exec_wrapper()` 执行完一条指令，便回调 `difftest_step()` 进行比对。在具体实现上，我们仅需补充寄存器比较这一部分的逻辑：

```

1 #define DIFF_REG(name) \
2     do { \
3         if (cpu.name != r.name) { \
4             printf("[DIFF] %-3s: \
5                 NEMU=0x%08x  QEMU=0x%08x  (at eip=0x%08x)\n", \
6                 #name, cpu.name, r.name, eip); \
7             diff = true; \
8         } \
9     } while (0)
10
11 DIFF_REG(eax);

```

```

12 DIFF_REG(ecx);
13 DIFF_REG(edx);
14 DIFF_REG(ebx);
15 DIFF_REG(esp);
16 DIFF_REG(ebp);
17 DIFF_REG(esi);
18 DIFF_REG(edi);
19 DIFF_REG(eip);

```

宏 `DIFF_REG` 利用 `#name` 把寄存器名变成字符串，便于打印；逐条比较 8 个通用寄存器与 `eip` 之后，若 `diff` 为真则将 `nemu_state` 置为 `NEMU_END`，让 NEMU 在下一轮 `cpu_exec` 时退出。在这里，EFLAGS 与内存均不参与比较：前者因为 NEMU 仅实现了 i386 中常用的若干位，与 QEMU 必然存在差异；后者则是因为遍历内存开销过大，同时也没什么必要。此外，框架还提供了 `is_skip_qemu` 与 `is_skip_nemu` 标志，用于跳过那些 NEMU 与 QEMU 行为本身就会不一致的指令（例如读取设备端口的 `in` 指令），在差异检测和正确性之间作出了一定的权衡。

启用 differential testing 只需在 `nemu/include/common.h` 中定义 `DIFF_TEST` 宏。需要注意的是，由于每条指令都要等待 QEMU 单步与回包，开启后 NEMU 的运行速度会大幅下降，因此，我们仅将它作为调试工具，只有需要的时候才开启宏启用这一模式，平时保持关闭以免影响开发与测试的效率。

ii. 条件码与 `jcc` / `setcc`

`cputest` 中的绝大多数程序都依赖条件分支，而条件分支的核心是“根据 EFLAGS 判断条件是否成立”。i386 共定义了 16 种条件码（cc），每种条件码对应一组 EFLAGS 标志位之间的逻辑关系；`jcc`、`setcc` 等指令在指令编码的低 4 位中携带条件码，执行时再根据条件码查询 EFLAGS 决定行为。NEMU 在 `cc.c` 中预先留好了 `rtl_setcc` 的框架，把条件码评估的实现集中在了同一个位置，使后续 `jcc`、`setcc` 可以共同使用该逻辑。

观察 16 种条件码的分布可以发现，奇数编码总是对应偶数编码的反义（如 `ne = !e`、`nb = !b`）。框架中依据此采取了如下办法：将 `subcode & 0xe` 截掉最低位，统一处理偶数情况；最后再根据 `subcode & 0x1` 决定是否进行一次 `xori 0x1` 的翻转。因此，我们实际需要完成的部分，只需为偶数条件码补全 8 个 case 即可：

```

1 rtlreg_t f1, f2;
2 switch (subcode & 0xe) {
3     case CC_0:  rtl_get_OF(dest); break;
4     case CC_B:  rtl_get_CF(dest); break;
5     case CC_E:  rtl_get_ZF(dest); break;
6     case CC_BE: rtl_get_CF(&f1); rtl_get_ZF(&f2);

```

```

7         rtl_or(dest, &f1, &f2); break;
8     case CC_S: rtl_get_SF(dest); break;
9     case CC_L: rtl_get_SF(&f1); rtl_get_OF(&f2);
10             rtl_xor(dest, &f1, &f2); break;
11     case CC_LE: rtl_get_SF(&f1); rtl_get_OF(&f2);
12             rtl_xor(dest, &f1, &f2);
13             rtl_get_ZF(&f2);
14             rtl_or(dest, dest, &f2); break;
15     default: panic("should not reach here");
16     case CC_P: panic("n86 does not have PF");
17 }

```

其中 `CC_B` 对应无符号比较的 `b` (below), 仅查 CF; `CC_L` 对应有符号比较的 `l` (less), 需要 `SF ^ OF`; `CC_BE` 与 `CC_LE` 则分别在两者基础上加上 `ZF`, 对应“小于等于”。这里所用的临时变量是函数局部的 `f1`、`f2`, 而不是 RTL 全局临时寄存器 `t0 - t3`。这是因为 `rtl_setcc` 会被 `exec_jcc` 调用, 而这时, 又常常仍需要 `t0 - t3` 中尚未失效的值, 借用全局寄存器会破坏调用上下文。

条件码这部分完成后, `jcc` 与 `setcc` 的实现就变得很简单。`jcc` 已经由框架给出, 它从 opcode 的低 4 位取 subcode, 调用 `rtl_setcc` 得到一个 0/1 的判断结果, 并将其作为 `decoding.is_jump` 的值:

```

1 make_EHelper(jcc) {
2     uint8_t subcode = decoding.opcode & 0xf;
3     rtl_setcc(&t2, subcode);
4     decoding.is_jump = t2;
5     print_asm("j%s %x", get_cc_name(subcode), decoding.jump_eip);
6 }

```

`setcc` 则将判断结果直接写入目标操作数 (一个字节), 用于把比较结果转换成布尔值。在 `opcode_table` 中只需将 `0x70--0x7f` (rel8 短跳转) 与 `0x0f80--0x0f8f` (rel32 长跳转) 的 16 个槽位都指向 `exec_jcc`, 将 `0x0f90--0x0f9f` 指向 `exec_setcc`, 就可以覆盖所有 16 种条件分支与赋值, 无需再单独额外编写。

iii. 指令的批量实现

完成 differential testing 与条件码之后, 接下来的部分就是依照测试结果来逐步补全指令。具体的开发流程与阶段一一致: 从 `cputest` 中未通过的测试开始, 运行后查看报错与反汇编, 实现所需的新指令, 然后再运行重复上述操作, 直到最终通过, 再进行下一个尚未通过的测试, 以避免一次性大量编码后导致难以判断错误位置的情况。

在先前的工作中, 已经实现了 RTL 的统一接口, 这里我们要实现的绝大多数 x86 指令在模式上较为相似。算术部分 `arith.c` (`add`、`sub`、`cmp`、`inc`、`dec`、`neg`、`adc`、`sbb`)

共享同一套“执行运算—写回结果—更新 EFLAGS”的逻辑结构，差别只在运算本身与标志位的计算细节；逻辑部分 `logic.c` (`and`、`or`、`xor`、`not`、`test`) 则相对更为简单，根据 i386 手册的规定，逻辑运算把 CF 与 OF 强制清零，仅需根据结果更新 ZF 与 SF 即可。例如 `and` 与 `or` 的实现：

```

1 make_EHelper(and) {
2     rtl_and(&t2, &id_dest->val, &id_src->val);
3     operand_write(id_dest, &t2);
4     rtl_li(&t0, 0);
5     rtl_set_CF(&t0);
6     rtl_set_OF(&t0);
7     rtl_update_ZFSF(&t2, id_dest->width);
8     print_asm_template2(and);
9 }
10
11 make_EHelper(or) {
12     rtl_or(&t2, &id_dest->val, &id_src->val);
13     operand_write(id_dest, &t2);
14     rtl_li(&t0, 0);
15     rtl_set_CF(&t0);
16     rtl_set_OF(&t0);
17     rtl_update_ZFSF(&t2, id_dest->width);
18     print_asm_template2(or);
19 }

```

两者唯一的差别是核心运算 `rtl_and` 与 `rtl_or`。而算术部分的 `cmp` 与 `test` 也有类似的相同模式，这两者与 `sub`、`and` 的标志更新逻辑一致，但差别在于不写回结果，只用于条件判断：

```

1 make_EHelper(cmp) {
2     rtl_sub(&t2, &id_dest->val, &id_src->val);
3     rtl_sltu(&t0, &id_dest->val, &id_src->val);
4     rtl_set_CF(&t0);
5     rtl_update_ZFSF(&t2, id_dest->width);
6
7     rtl_xor(&t0, &id_dest->val, &id_src->val);
8     rtl_xor(&t1, &id_dest->val, &t2);
9     rtl_and(&t0, &t0, &t1);
10    rtl_msb(&t0, &t0, id_dest->width);
11    rtl_set_OF(&t0);
12

```

```

13     print_asm_template2(cmp);
14 }

```

inc 与 dec 看上去与 add、sub 类似，但 i386 手册有明确规定它们不修改 CF。这是 inc 与 add 1 的唯一的区别，也是不能用 `rtl_add` + `rtl_set_CF` 实现 inc 的根本原因。此外，由于 inc 只在“原符号 = 0 且新符号 = 1”时发生溢出，OF 的计算可以从 add 的通用形式进行简化得到：

```

1  make_EHelper(inc) {
2      rtl_addi(&t2, &id_dest->val, 1);
3
4      rtl_msb(&t0, &t2, id_dest->width);
5      rtl_msb(&t1, &id_dest->val, id_dest->width);
6      rtl_not(&t1);
7      rtl_and(&t0, &t0, &t1);
8      rtl_set_OF(&t0);
9
10     operand_write(id_dest, &t2);
11     rtl_update_ZFSF(&t2, id_dest->width);
12     print_asm_template1(inc);
13 }

```

数据传输 data-move.c 这部分指令的实现则有所不同。一部分指令在 RTL 层面就能完成（如 lea、leave、movsx、movzx），在框架中已经实现了这些函数，我们只需在 opcode_table 中注册即可。另一部分指令则需要利用 cltd、cwtl 这类扩展的逻辑，需要配合 `rtl_msb`、`rtl_sub` 等基本指令完成。这里我们以 cltd 的实现为例：

```

1  make_EHelper(cltd) {
2      if (decoding.is_operand_size_16) {
3          rtl_lr_w(&t0, R_AX);
4          rtl_sext(&t0, &t0, 2);
5          rtl_sr_l(R_EAX, &t0);
6      }
7      else {
8          rtl_lr_l(&t0, R_EAX);
9          rtl_msb(&t0, &t0, 4);
10         rtl_sub(&t0, &tzero, &t0);
11         rtl_sr_l(R_EDX, &t0);
12     }
13     print_asm(decoding.is_operand_size_16 ? "cwtl" : "cltd");
14 }

```

其中 `rtl_sub(&t0, &tzero, &t0)` 利用了补码的性质: $0 - 0 = 0$, $0 - 1 = -1 = 0xFFFF$ 。这样的写法恰好把 EAX 的符号位扩展为对 EDX 的全 0 或全 1 填充, 避免了使用条件分支。`tzero` 是 RTL 中的“0 寄存器”, 永远读出 0, 正是为这种场合而设计的。

控制 `control.c` 这部分的指令在前一阶段已经实现了 `call`、`ret`、`jmp` 的直接形式, 这里我们还需要补全间接形式 `call_rm` 与 `jmp_rm`: 它们的目标地址来自寄存器或内存而非立即数, 运算时只需把 `id_dest->val` 作为目标地址即可:

```
1 make_EHelper(call_rm) {
2     rtl_li(&t0, decoding.seq_eip);
3     rtl_push(&t0);
4     decoding.jump_eip = id_dest->val;
5     decoding.is_jump = 1;
6     print_asm("call %s", id_dest->str);
7 }
```

这类指令在实际应用中也很常见, C 语言中的函数指针调用 `f(x)` 在编译后即对应一条 `call r/m32` 指令; 递归测试 `recursion` 中也使用函数指针表来实现递归的调度。

最后是移位指令 `shl`、`shr`、`sar`, 它们的核心运算逻辑 `rtl_shl` 等已经在框架中写好了。需要注意的是, 由于 NEMU 简化了 EFLAGS 的更新逻辑, 移位指令在我们的实现中不会修改 CF 与 OF: 在实验文档中也有所提及, 虽然有所省略, 但是对当前的 `cputest` 与 PA2 后续的操作并不会产生影响。

iv. 测试与调试

依照上述流程推进 `cputest` 时, 也会出现各种其他的报错。在实际的调试中, 受制于开发环境, 无法使用类似于集成开发环境 (如 Visual Studio 等) 中的断点功能。除了开启 `diff-test` 之外, 使用打印输出来进行 debug 也是一种简便的方法, 我们可以在某一步的函数中添加一段打印的代码, 从而在执行过程中查看每一步对应各个变量当前的值。

在 `leap-year` 这部分的测试中, NEMU 没有报告 `invalid opcode`, 而是直接被 `SIGFPE` (Floating point exception) 报错退出。这里我们可以得知, 崩溃发生在 `rtl_idiv` 内联汇编真正执行硬件 `idiv` 指令的那一刻, 而非来自程序的语义逻辑错误。

最初的怀疑是 `cld` 实现错误, 未能正确把 EDX 清零; 其次是 `exec_idiv` 中操作数的某一步取错了寄存器。但仅靠人工审阅代码, 三条假设都无法证伪。这时唯一可靠的办法是在崩溃发生前的最后一刻打印现场。于是在 `exec_idiv` 入口处插入了诊断输出:

```
1 fprintf(stderr,
2     "[IDIV] eip=0x%08x  eax=0x%08x  edx=0x%08x  divisor=0x%08x\n",
```

```
3   cpu.eip, dbg_eax, dbg_edx, id_dest->val);
```

再次运行后得到输出：

```
1  [IDIV] eip=0x001000dc  eax=0x00000762  edx=0x00000000  divisor=0
    x00000000
```

分析可知，eax 是 0x762 (1890，正是闰年判断的起始年份)，edx 是 0 (cltd 工作正常)，问题在于 divisor 的值竟然是 0。然而查看反汇编，显示这条指令却是 `idiv %esi`，而 esi 在 main 入口处又应被 `mov $0x190, %esi` 装载为 400。于是我们继续在 `exec_mov` 代码中加上了类似的打印，经过测试证实了 `mov $0x190, %esi` 在到达 `idiv` 之前确实正确执行过。通过这些信息我们可以判断出，问题并不在于 esi，而是 `idiv` 在执行时根本没读到 esi 的值。

这时再回到 `exec.c`，观察 gp3 子表的注册：

```
1  make_group(gp3,
2      IDEX(test_I, test), EMPTY, EMPTY, EMPTY,
3      IDEX(E, mul), IDEX(E, imul1), IDEX(E, div), IDEX(E, idiv))
```

这里的写法看似自然，却正是 bug 的来源。gp3 在主表中通过 `IDEX(E, gp3)` 进入：主表层已经调用了一次 `decode_op_rm`，把 ModR/M 字节解析完毕，esi 的值也已被读入 `id_dest->val`。然后控制权交到 gp3 子表，但子表中再次使用 `IDEX(E, ...)`，意味着 `decode_op_rm` 会被第二次调用。此时 eip 已经走过 ModR/M 字节，指向之后的内容（对于 `idiv` 来说是下一条指令的开头），第二次译码于是从错误的位置读取字节、生成错误的操作数，把 `id_dest` 整体覆写——最终 `idiv` 接收到的“操作数”实际上是从乱码字节中解析出的一个 0。回看 gp1 与 gp5 子表，框架代码里 `mul`、`push` 等子项都使用的是不再次译码的 `EX(...)`，正是为了避免同样的问题。gp3 是个混合体：/0 位置的 `test` 因为还需要再读一个立即数，必须保留 `IDEX(test_I, test)`；而 /4-/7 的 `mul`、`imul`、`div`、`idiv` 不再需要任何字节，应当使用 `EX(...)` 而非 `IDEX(...)`。

修复后的 gp3 子表如下：

```
1  make_group(gp3,
2      IDEX(test_I, test), EMPTY, EMPTY, EMPTY,
3      EX(mul), EX(imul1), EX(div), EX(idiv))
```

重新编译运行，`leap-year` 立即通过测试。这里我们通过上述调试的过程可以总结一些调试经验：可以通过控制台打印的方式来获取程序崩溃前每一步各个变量的状态，从中查看反常的值并逐步定位报错的来源；同时，框架中看似一致的写法也未必真的一致，子表中 `IDEX` 与 `EX` 的选择，本质取决于该指令是否仍需读取后续字节；这一点在实验文档中并没有被直接说明，需要观察代码中的提示并总结报错这些问题，才能辨别 gp1 / gp3 / gp5 这三个表之间在设计上的细微差别。

后续的流程较为顺利，通过以上的办法，可以高效地找出报错的根源。经过不断的测试与调试，cputest 中的全部 32 个测试程序在 NEMU 上均能正确通过，最后使用 `runall.sh` 进行一键回归测试，结果如图 1 所示。

```
dev@ubuntu:~/ics2017/nexus-am/tests/cputest$ cd ~/ics2017/nemu
dev@ubuntu:~/ics2017/nemu$ bash runall.sh
NEMU compile OK
compiling testcases...
testcases compile OK
[ add-longlong] PASS!
[ add] PASS!
[ bit] PASS!
[ bubble-sort] PASS!
[ dummy] PASS!
[ fact] PASS!
[ fib] PASS!
[ goldbach] PASS!
[ hello-str] PASS!
[ if-else] PASS!
[ leap-year] PASS!
[ load-store] PASS!
[ matrix-mul] PASS!
[ max] PASS!
[ min3] PASS!
[ mov-c] PASS!
[ movsx] PASS!
[ mul-longlong] PASS!
[ pascal] PASS!
[ prime] PASS!
[ quick-sort] PASS!
[ recursion] PASS!
[ select-sort] PASS!
[ shift] PASS!
[ shuixianhua] PASS!
[ string] PASS!
[ sub-longlong] PASS!
[ sum] PASS!
[ switch] PASS!
[ to-lower-case] PASS!
[ unalign] PASS!
[ wanshu] PASS!
dev@ubuntu:~/ics2017/nemu$
```

图 1: cputest 测试结果

到此为止，cputest 这一阶段实现完成。在这一阶段，主要采取了按需推进的思路进行实现，相较于一次性写完所有指令，这样逐步进行要更加稳健，每实现一两条指令就运行一次测试，使得错误位置也能更加方便地限定在最近的改动范围内。

遗憾的是，这次的流程中并未用到 differential testing，测试中并未出现难以解决的潜在问题。但是我们可以想象，这种推进的模式实际上也很有利于充分发挥 differential testing 的功能：当某个 cputest 顺利通过时，其实并不意味着相关的指令完全正确，只是当前测试未触发其潜在的 bug；而 differential testing 则能在测试通过但实现有问题时提供决定性的线索，从而定位测试中问题的根源。

4. 输入输出与 AM

通过前面两个阶段，现在的 NEMU 已经可以正确执行各种纯计算的 x86 程序，但它仍然没有任何的交互，没有键盘输入也没有任何形式的输出。在 PA2 的最后一个阶段，我们还需为 NEMU 实现输入与输出，使其成为一台真正意义上的完整的计算机。

NEMU 通过两种方式与设备交互：端口映射 I/O 使用专用的 in、out 指令访问设备寄存器；内存映射 I/O 则将设备寄存器或缓冲区映射到一段特定的物理地址，对该地址的普通访存即对应相应的设备访问。串口、键盘、RTC 时钟采用端口映射 I/O，VGA 显存采用内存映射 I/O。两者在框架中都已经实现了底层模拟，我们还需把 CPU 这一侧的 in/out 指令、物理地址访存与下层 `pio_read/write`、`mmio_read/write` 衔接起来；以及在 AM 部分，向上提供 `_putc`、`_uptime`、`_read_key`、`_draw_rect` 等抽象接口，以供应用程序来调用。

i. in / out

所有经由 PIO 的设备访问最终都通过 in 与 out 完成。实验框架中已经在 `decode.c` 中实现了四个译码 helper (`in_I2a`、`in_dx2a`、`out_a2I`、`out_a2dx`)，负责把端口号与数据正确分配到 `id_src` 与 `id_dest` 中；我们只需在执行阶段调用 `pio_read` 与 `pio_write`：

```

1 make_EHelper(in) {
2     uint32_t data = pio_read(id_src->val, id_dest->width);
3     rtl_li(&t0, data);
4     operand_write(id_dest, &t0);
5     print_asm_template2(in);
6     #ifdef DIFF_TEST
7         diff_test_skip_qemu();
8     #endif
9 }
10
11 make_EHelper(out) {
12     pio_write(id_dest->val, id_src->width, id_src->val);
13     print_asm_template2(out);
14     #ifdef DIFF_TEST
15         diff_test_skip_qemu();
16     #endif
17 }
```

in 的端口号位于 `id_src` (结果写到 `id_dest`)，out 的端口号则位于 `id_dest` (数据来自 `id_src`)。这与 AT&T 汇编中 in 的源/目的写法保持一致。此外，由于 NEMU 中的串口模拟做了简化，状态寄存器始终保持就绪，QEMU 的真实串口实现与这里并不一定相同，所以这两条指令在 `DIFF_TEST` 启用时都调用 `diff_test_skip_qemu()`，来让 differential testing 跳过这一条指令避免报告差异。

ii. HAS_IOE 与 Hello World

在 `nemu/include/common.h` 中定义 `HAS_IOE`，使 NEMU 在 `init_monitor` 时调用 `init_device()`，注册串口、键盘、定时器、VGA 等设备；同时在 `cpu_exec` 的每轮主循环中调用 `device_update`，让设备状态以约 100 Hz 的频率得到刷新。AM 一侧则需在 `trm.c` 中定义 `HAS_SERIAL`，使 `_putc` 通过 `outb` 指令将字符发送到串口的 0x3f8 端口。运行 `nexus-am/apps/hello`，可以在终端中看到连续十行 `Hello World!` 后程序正常返回。

```
dev@ubuntu:~/ics2017/nemu$ cd ~/ics2017/nexus-am/apps/hello
dev@ubuntu:~/ics2017/nexus-am/apps/hello$ make ARCH=x86-nemu run
Building hello [x86-nemu]
+ CC hello.c
make[1]: Entering directory '/home/dev/ics2017/nexus-am'
make[2]: Entering directory '/home/dev/ics2017/nexus-am/am'
Building am [x86-nemu]
+ CC arch/x86-nemu/src/trm.c
+ AR /home/dev/ics2017/nexus-am/am/build/am-x86-nemu.a
make[2]: Leaving directory '/home/dev/ics2017/nexus-am/am'
make[1]: Leaving directory '/home/dev/ics2017/nexus-am'
make[1]: Entering directory '/home/dev/ics2017/nexus-am/libs/klib'
make[1]: *** No targets specified and no makefile found. Stop.
make[1]: Leaving directory '/home/dev/ics2017/nexus-am/libs/klib'
/home/dev/ics2017/nexus-am/Makefile.compile:86: recipe for target 'klib' failed
make: [klib] Error 2 (ignored)
make[1]: Entering directory '/home/dev/ics2017/nemu'
./build/nemu -l /home/dev/ics2017/nexus-am/apps/hello/build/nemu-log.txt /home/dev/ics2017/nexus-am/apps/hello/build/hello-x86-nemu.bin
[src/monitor/monitor.c,65,load_img] The image is /home/dev/ics2017/nexus-am/apps/hello/build/hello-x86-nemu.bin
Welcome to NEMU!
[src/monitor/monitor.c,30,welcome] Build time: 02:23:24, May 17 2026
For help, type "help"
(nemu) c
Hello World!
Hello World!
Hello World!
Hello World!
Hello World!
Hello World!
Hello World!
Hello World!
Hello World!
Hello World!
nemu: HIT GOOD TRAP at eip = 0x0010006e
(nemu) q
make[1]: Leaving directory '/home/dev/ics2017/nemu'
```

图 2: Hello World 测试结果

在具体流程上，应用调用 `_putc('H')`，AM 层翻译为 `outb 0x3f8, %al`；NEMU 中 `exec_out` 被调用，转而调用 `pio_write`；`pio_write` 查端口表找到串口的回调函数；回调将 `'H'` 写到主机的标准输出。这一链路由“程序—AM—CPU—设备—主机”构成，虽然代码很少，但却是这一阶段最为关键的部分。

iii. 时钟与键盘

时钟与键盘也都基于端口 I/O。NEMU 模拟了一个非标准的 RTC，会记录 NEMU 启动到现在的毫秒数放在端口 0x48，4 字节读取；AM 在 `_ioe_init` 时记下 `boot_time`，`_uptime` 用当前的值减去启动时的值，就可以得到从 AM 启动以来的毫秒数：

```
1 unsigned long _uptime() {
2     return inl(RTC_PORT) - boot_time;
3 }
```

键盘则模拟 i8042 控制器：0x64 是状态端口，最低位为 1 表示有按键事件等待读取；0x60 是数据端口，按下按键时返回扫描码加 0x8000，释放按键时返回原始扫描码。

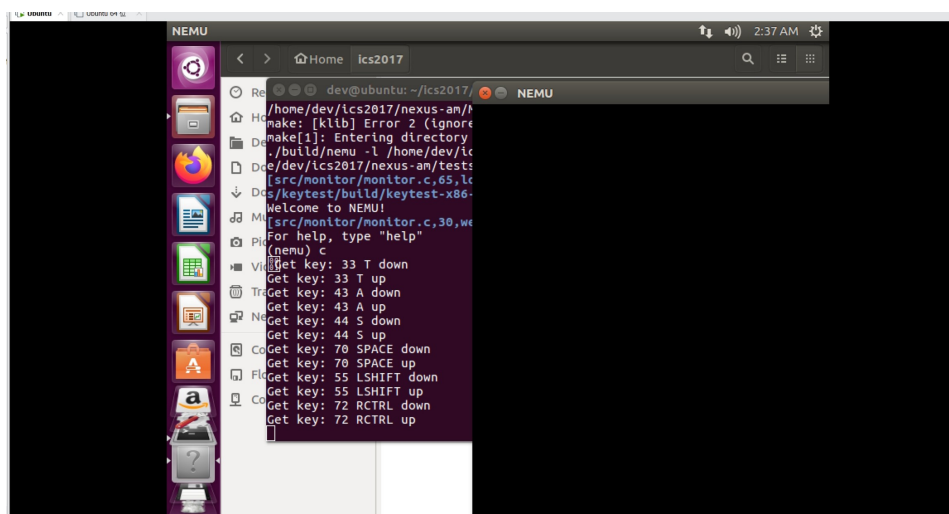
`_read_key` 的实现如下:

```
1 int _read_key() {
2     if (inb(0x64) & 0x01) {
3         return inl(0x60);
4     }
5     return _KEY_NONE;
6 }
```

完成上述内容后, `tests/timetest` 每秒在终端输出一行 `N seconds.`, 按规律递增; `tests/keytest` 启动后, 在弹出的 VGA 窗口中按下与释放任意按键, 终端立即打印 `Get key: NN <NAME> down/up`。两个测试的结果都符合预期。

```
dev@ubuntu:~/ics2017/nexus-am/tests/timetest$ make ARCH=x86-nemu run
Building timetest [x86-nemu]
make[1]: Entering directory '/home/dev/ics2017/nexus-am'
make[2]: Entering directory '/home/dev/ics2017/nexus-am/am'
Building am [x86-nemu]
make[2]: Nothing to be done for 'archive'.
make[2]: Leaving directory '/home/dev/ics2017/nexus-am/am'
make[1]: Leaving directory '/home/dev/ics2017/nexus-am'
make[1]: Entering directory '/home/dev/ics2017/nexus-am/libs/klib'
make[1]: *** No targets specified and no makefile found. Stop.
make[1]: Leaving directory '/home/dev/ics2017/nexus-am/libs/klib'
/home/dev/ics2017/nexus-am/Makefile.compile:86: recipe for target 'klib' failed
make: [klib] Error 2 (ignored)
make[1]: Entering directory '/home/dev/ics2017/nemu'
./build/nemu -l /home/dev/ics2017/nexus-am/tests/timetest/build/nemu-log.txt /home/dev/ics2017/nexus-am/tests/timetest/build/timetest-x86-nemu.bin
[src/monitor/monitor.c,65,load_img] The image is /home/dev/ics2017/nexus-am/tests/timetest/build/timetest-x86-nemu.bin
Welcome to NEMU!
[src/monitor/monitor.c,30,welcome] Build time: 02:23:24, May 17 2026
For help, type "help"
(nemu) c
1 second.
2 seconds.
3 seconds.
4 seconds.
5 seconds.
6 seconds.
7 seconds.
8 seconds.
9 seconds.
10 seconds.
^C/home/dev/ics2017/nexus-am/Makefile.app:35: recipe for target 'run' failed
make: *** [run] Interrupt
dev@ubuntu:~/ics2017/nexus-am/tests/timetest$
```

图 3: timetest 测试结果



```

NEMU
dev@ubuntu:~/ics2017/nexus-am/tests/keytest$ make ARCH=x86-nemu run
Building keytest [x86-nemu]
make[1]: Entering directory '/home/dev/ics2017/nexus-am'
make[2]: Entering directory '/home/dev/ics2017/nexus-am/am'
Building am [x86-nemu]
make[2]: Nothing to be done for 'archive'.
make[2]: Leaving directory '/home/dev/ics2017/nexus-am/am'
make[1]: Leaving directory '/home/dev/ics2017/nexus-am'
make[1]: Entering directory '/home/dev/ics2017/nexus-am/libs/klib'
make[1]: *** No targets specified and no makefile found. Stop.
make[1]: Leaving directory '/home/dev/ics2017/nexus-am/libs/klib'
/home/dev/ics2017/nexus-am/Makefile.compile:86: recipe for target 'klib' failed
make: [klib] Error 2 (ignored)
make[1]: Entering directory '/home/dev/ics2017/nemu'
./build/nemu -l /home/dev/ics2017/nexus-am/tests/keytest/build/nemu-log.txt /home/dev/ics2017/nexus-am/tests/keytest/build/keytest-x86-nemu.bin
[src/monitor/monitor.c,65,load_img] The image is /home/dev/ics2017/nexus-am/tests/keytest/build/keytest-x86-nemu.bin
Welcome to NEMU!
[src/monitor/monitor.c,30,welcome] Build time: 02:23:24, May 17 2026
For help, type "help"
(nemu) c
Get key: 33 T down
Get key: 33 T up
Get key: 43 A down
Get key: 43 A up
Get key: 44 S down
Get key: 44 S up
Get key: 70 SPACE down
Get key: 70 SPACE up
Get key: 55 LSHIFT down
Get key: 55 LSHIFT up
Get key: 72 RCTRL down
Get key: 72 RCTRL up

```

图 4: keytest 测试结果

iv. VGA 与内存映射 I/O

在 VGA 这部分则与前面三个设备的访问有所不同。NEMU 将物理地址区间 `[0x40000, 0x80000]` 映射到 400×300 的 32 位帧缓冲，对这段地址的普通读写即等同于读写显存。为此，需要在 `paddr_read` 与 `paddr_write` 中加入对 MMIO 区域的判断，调用 `is_mmio` 检查地址是否落在某个 MMIO 区间，若是则改为 `mmio_read/write`，若不是就继续进行普通内存的访问：

```

1 uint32_t paddr_read(paddr_t addr, int len) {
2     int map_NO = is_mmio(addr);
3     if (map_NO != -1) {
4         return mmio_read(addr, len, map_NO);
5     }
6     return pmem_rw(addr, uint32_t) & (~0u >> ((4 - len) << 3));
7 }

```

`paddr_write` 的修改与之对称。从 CPU 视角看，访存指令并没有什么变化，但当地址落入 MMIO 区间时，这一访问会被透明地转发给 VGA 模拟。

在 AM 部分，`_draw_rect` 负责把一个 `w × h` 的像素小块复制到帧缓冲的指定位置。帧缓冲是一个 400×300 的二维数组，因此必须逐行进行复制，不能直接使用 `memcpy` 把整块数据当作连续内存复制：

```

1 void _draw_rect(const uint32_t *pixels, int x, int y, int w, int h) {
2     int row;
3     for (row = 0; row < h; row++) {
4         memcpy(&fb[(y + row) * _screen.width + x],
5              &pixels[row * w],
6              w * sizeof(uint32_t));
7     }
8 }

```

完成上述实现后，运行 `tests/videotest` 可以在 VGA 窗口中看到一组同心嵌套、颜色规则变化的矩形环，与设计预期完全一致。验证了上述代码的正确性。

v. 跑分

完成 IOE 之后，按照实验文档要求最后在 Dhrystone、CoreMark、microbench 三组程序上跑分。前两部分顺利完成，但在 microbench 过程中，10 个子项前 9 个可以正常通过，但在 md5 子项却触发了 `invalid opcode` 保持。查看相关的反汇编文件可知，md5 的核心运算使用了大量的 `rol` 循环左移指令，而这并不在 PA2 这部分中提及，框架也没有为它预留任何 helper 函数或 RTL 接口。

在这里，为了让 microbench 完整通过，最终在框架之外补全了 `rol` 与 `ror` 这两条指令。实际实现上，主要通过 `rtl_shl` 与 `rtl_shr` 组合进行实现：

```

1 make_EHelper(rol) {
2     int width_bits = id_dest->width * 8;
3     uint32_t n = id_src->val & (width_bits - 1);
4     if (n == 0) {
5         print_asm_template2(rol);
6         return;
7     }
8     rtl_li(&t0, n);
9     rtl_shl(&t1, &id_dest->val, &t0);
10    rtl_li(&t3, width_bits - n);
11    rtl_shr(&t2, &id_dest->val, &t3);
12    rtl_or(&t2, &t1, &t2);
13    operand_write(id_dest, &t2);
14    print_asm_template2(rol);
15 }

```

当 `n` 为 0 时 `width_bits - n` 等于位宽本身，对一个 `uint32_t` 移位 32 位在 C 标准中是未定义行为，大多数 x86 主机会把移位量按位掩码到 31，导致结果错误，因而这里对于 `n==0` 作提前的判断与处理。`ror` 也需要进行类似的处理。完成这两条指令后，microbench 的 10 个子项全部通过。再次运行这几个跑分程序进行测试，最终得到完整的测试结果。

Dhrystone 得分 59，CoreMark 得分 207，microbench 综合得分 230（10 个子项全部通过）。分数以 Intel i7-6700 @ 3.40GHz 真机为参照基准 100000，可以看到 NEMU 解释执行的速度大约是真机的 0.06%–0.23%，这一结果相对比较符合实验的预期。纯解释执行的模拟器每条客户指令都要经过取指、译码、执行、标志更新等若干步主机指令配合，性能开销极大。要想进一步提升性能，可行的方向是引入即时编译（JIT），把热路径上的客户指令翻译为本地指令并缓存执行，但这已经超出了 PA2 的范围。

截至到这里，PA2 的所有目标均已达成，我们终于得到了一个完整的 NEMU。完成实验流程后，我们尝试了实验文档中提及的 typing 小游戏，在 NEMU 中也能正常运行。



图 5: typing 游戏

游戏依赖键盘事件读取、VGA 帧绘制、时钟节拍三类 I/O 子系统同时工作，是对前面所有实现的一次综合检验。

二、总结与体会

回顾整个 PA2 的实现过程，主要采取了分阶段 + 按需推进这一开发的节奏进行实现。三个阶段分别有清晰的实现目标：阶段一实现 dummy 的运行，阶段二完成 cputest 部分的测试，阶段三对应 hello/timetest/keytest/videotest 的全面测试。每一个阶段都需要前面所有工作的支撑，同时只需要验证其中最少的必要的一小部分新代码。这样的流程让可能报错的空间始终被限制在最近一两次改动之内，几乎所有 bug 都能快速地找到问题根源，大大降低了调试的复杂程度。

综合来看，PA2 让我们把一台只能取指执行少数指令的简化图灵机，逐步扩充为一台能够运行真实的 C 程序、能够读取键盘显示图像的完整冯诺依曼计算机。这一过程虽然以 x86 整数子集为目标，但所涉及的每一个概念都是真实计算机系统中通用而核心的存在。完成 PA2 的后，再回过头看我们的计算机系统，也有了完全不同的认知，在自己一步步实现底层指令的过程中，对于系统的功能逻辑有了更加深入的理解，也锻炼了我的系统思维与工程理解。

三、必答题

(1).static 与 inline 的作用

选取 `rtl_li` 作为实验对象，分三种情况修改 `rtl.h` 后重新编译，观察到的结果与理论分析如下。

只去掉 inline，保留 static：编译报错，错误信息为

```
1 ./include/cpu/rtl.h:11:13: error: 'rtl_li' defined but not used
2   [-Werror=unused-function]
```

去掉 `inline` 之后，`rtl_li` 不再被展开到调用点，而是作为一个真正的函数存在于每个翻译单元中。由于 `static` 仍在，它只在本文件内可见。但 `watchpoint.c` 等文件 include 了 `rtl.h` 却并不调用 `rtl_li`，编译器发现“定义了但从未使用”，在 `-Werror` 的作用下这一警告被提升为错误，导致编译失败。

只去掉 static，保留 inline：编译通过，未见报错。

这个结果源于 C 对 `inline` 函数的特殊规定：若一个函数仅声明为 `inline` 而未提供外部链接的非内联定义，编译器可以选择在每个调用点内联展开而不产生独立的外部符号。GCC 在当前编译选项下也正是如此处理的。所有翻译单元都把 `rtl_li` 内联了，链接器从未看到重复的外部符号，编译因此通过。然而这一行为依赖于具体的编译器版本与优化级别：若换用更高优化级别或其他编译器，可能选择不内联而暴露外部符号，届时就会出现链接错误。换言之，去掉 `static` 仅靠 `inline` 虽然在这里正确，但是并不严谨，`static` 的存在正是为了消除这种对编译器行为的依赖。

两者都去掉：链接报错，错误信息为

```
1 build/obj/cpu/exec/special.o: In function `rtl_li':
2   rtl.h:12: multiple definition of `rtl_li'
3   build/obj/monitor/debug/watchpoint.o:
4   rtl.h:12: first defined here
5   ... (后续每个 .o 均重复报同样错误)
```

既没有 `inline` 阻止产生独立符号，也没有 `static` 限制链接可见性，每一个 include 了 `rtl.h` 的 `.c` 文件都生成了一份全局可见的 `rtl_li` 定义。链接器在合并所有 `.o` 时发现同名外部符号出现多次，从而导致报 `multiple definition` 错误。

综合三次实验：`inline` 的作用是让编译器将函数体展开到调用点，避免产生独立的外部符号；`static` 的作用是将符号的链接可见性限制在本翻译单元之内。两者共同作用，才能保证定义在头文件中的 RTL 指令函数既不会因定义了没调用被 `-Werror` 拦截，也不会因多份定义共存被链接器拒绝。这与实验中直接观察到的三种报错现象完全吻合，也证明了两个关键字各自的职责不能相互替代，缺一不可。

(2).Makefile 的工作机制与 NEMU 的编译链接过程

在 `nemu` 目录下执行 `make`，Makefile 大致按以下流程组织：首先通过 `find` 在源码目录中扫描所有 `.c` 文件并展开为对应的 `.o` 目标列表；接着对每一个 `.o` 目标，根据 implicit rule `$(BUILD_DIR)/%.o: %.c` 调用 GCC 编译，输出到 `build/obj/` 下；最后将所有 `.o` 链接为可执行文件 `build/nemu`。

Makefile 中使用了多种特性来组织这一过程。变量 `CC`、`CFLAGS`、`LDFLAGS` 等被定义在文件头部，便于一处修改影响全局；`INC_DIR` 与 `-I` 选项使 `include/` 与各子模块下的头文件可以被正确找到；`$(call ...)` 与 `$(wildcard ...)` 等函数式特性被用于动态展开文件列表。Makefile 还通过 `include` 指令将 `Makefile.git`、`Makefile.compile` 等子文件引入主文件，实现关注点分离——例如编译规则与 git 钩子的处理被分置在不同的子文件中。

implicit rule（隐式规则）是 GNU Make 的内建机制，对“源文件—目标文件”的常见模式预先提供了构建规则。例如 `%.o: %.c` 是一条匹配规则，告诉 Make 当目标是 `x.o`、依赖是 `x.c` 时如何调用编译器。NEMU 的 Makefile 重写了这一隐式规则，加入了自定义的输出路径、依赖文件生成（`-MMD -MP`）、彩色提示等行为，但其核心模式与 GNU 默认提供的规则一脉相承。

要观察 Makefile 实际执行了哪些命令，可以使用 `make -n` 选项进行“dry run”：它会打印将要执行的命令而不真正执行。对 NEMU 执行 `make -n` 可以看到一长串 `gcc -c ... -o build/obj/xxx.o xxx.c` 的编译命令，最后是一条 `gcc -o build/nemu build/o` 的链接命令。结合 `-MMD -MP` 生成的 `.d` 依赖文件，Make 知道每个 `.o` 究竟依赖哪些头文件，从而在头文件变化时也能正确触发增量编译。

编译链接的过程本身分为四个阶段：预处理（`cpp`）展开宏与 `#include`、生成纯净的 C 源代码；编译（`cc1`）将其翻译为汇编代码；汇编（`as`）将汇编翻译为目标文件中的机器码与符号表；链接（`ld`）将所有目标文件以及标准库合并，生成最终的可执行文件。NEMU 的 `build/nemu` 即是这四个阶段相继作用的产物。理解这一流程后，再回头看 Makefile 中各项变量与规则的设计，就能清楚地把它们映射到这条工具链上的某一环节。